



Download infrastructure in Buildroot





Introduction

- ▶ One important aspect of Buildroot is to fetch source code or binary files from third party projects.
- ▶ Download supported from HTTP(S), FTP, Git, Subversion, CVS, Mercurial, etc.
- ▶ Being able to do reproducible builds over a long period of time requires understanding the download infrastructure.



Download location

- ▶ Each Buildroot package indicates in its `.mk` file which files it needs to be downloaded.
- ▶ Can be a tarball, one or several patches, binary files, etc.
- ▶ When downloading a file, Buildroot will successively try the following locations:
 1. The local `$(DL_DIR)` directory where downloaded files are kept
 2. The **primary site**, as indicated by `BR2_PRIMARY_SITE`
 3. The **original site**, as indicated by the package `.mk` file
 4. The **backup Buildroot mirror**, as indicated by `BR2_BACKUP_SITE`



- ▶ Once a file has been downloaded by Buildroot, it is cached in the directory pointed by `$(DL_DIR)`, in a sub-directory named after the package.
- ▶ By default, `$(TOPDIR)/dl`
- ▶ Can be changed
 - using the `BR2_DL_DIR` configuration option
 - or by passing the `BR2_DL_DIR` environment variable, which overrides the config option of the same name
- ▶ The download mechanism is written in a way that allows independent parallel builds to share the same `DL_DIR` (using atomic renaming of files)
- ▶ No cleanup mechanism: files are only added, never removed, even when the package version is updated.



Primary site

- ▶ The `BR2_PRIMARY_SITE` option allows to define the location of a HTTP or FTP server.
- ▶ By default empty, so this feature is disabled.
- ▶ When defined, used in priority over the original location.
- ▶ Allows to do a local mirror, in your company, of all the files that Buildroot needs to download.
- ▶ When option `BR2_PRIMARY_SITE_ONLY` is enabled, only the *primary site* is used
 - It does not fall back on the original site and the backup Buildroot mirror
 - Guarantees that all downloads must be in the primary site



Backup Buildroot mirror

- ▶ Since sometimes the upstream locations disappear or are temporarily unavailable, having a backup server is useful
- ▶ Address configured through `BR2_BACKUP_SITE`
- ▶ Defaults to <http://sources.buildroot.net>
 - maintained by the Buildroot community
 - updated before every Buildroot release to contain the downloaded files for all packages
 - exception: cannot store all possible versions for packages that have their version as a configuration option. Generally only affects the kernel or bootloader, which typically don't disappear upstream.



Special case of VCS download

- ▶ When a package uses the source code from Git, Subversion or another VCS, Buildroot cannot directly download a tarball.
- ▶ It uses a VCS-specific method to fetch the specified version of the source from the VCS repository
- ▶ The source code is checked-out/cloned inside `DL_DIR` and kept to speed-up further downloads of the same project (caching only implemented for Git)
- ▶ Finally a tarball containing only the source code (and not the version control history or metadata) is created and stored in `DL_DIR`
 - Example: `c-capnproto-9053ebe6eeb2ae762655b982e27c341cb568366d-git4.tar.gz`
- ▶ This tarball will be re-used for the next builds, and attempts are made to download it from the primary and backup sites.
- ▶ Due to this, always use a tag name or a full commit id, and never a branch name: the code will never be re-downloaded when the branch is updated.



Vendoring

- ▶ Some language-specific package management systems like to download the dependencies by themselves: *vendoring*
- ▶ Examples: *Cargo* in the Rust ecosystem, or *Go*
- ▶ Problem for build systems: reproducibility of the builds, licensing, offline builds
- ▶ Buildroot supports vendoring dependencies for *cargo* and *go* packages
- ▶ Right after the download of the package source code, Buildroot invokes the language-specific vendoring tool, and bundles the dependencies inside the tarball



File integrity checking

- ▶ Buildroot packages can provide a `.hash` file to provide *hashes* for the downloaded files.
- ▶ The download infrastructure uses this hash file when available to check the integrity of the downloaded files.
- ▶ Hashes are checked every time a downloaded file is used, even if it is already cached in `$(DL_DIR)`.
- ▶ If the hash is incorrect, the download infrastructure attempts to re-download the file once. If that still fails, the build aborts with an error.

Hash checking message

```
strace-4.10.tar.xz: OK (md5: 107a5be455493861189e9b57a3a51912)
strace-4.10.tar.xz: OK (sha1: 5c3ec4c5a9eeb440d7ec70514923c2e7e7f9ab6c)
>>> strace 4.10 Extracting
```



Download-related `make` targets

▶ `make source`

- Triggers the download of all the files needed to build the current configuration.
- All files are stored in `$(DL_DIR)`
- Allows to prepare a fully offline build

▶ `make external-deps`

- Lists the files from `$(DL_DIR)` that are needed for the current configuration to build.
- Does not guarantee that all files are in `$(DL_DIR)`, a `make source` is required